

ANÁLISIS DEL RETO

John Acosta, 202212004, ja.acostar1@uniandes.edu.co

Andres Zamora, 202516126, as.zamorar1@uniandes.edu.co

La repartición del trabajo fue: John Acosta (Requerimiento 1), Andrés Zamora (Carga de datos y requerimiento 2), y requerimiento 4,5 y 6 en conjunto. Debido a que en el documento del reto no se cuenta la carga de datos como requerimiento, no incluimos su análisis.

Requerimiento <<n>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado (Sí/No)	Si se implementó y quien lo hizo.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1	$O(\dots)$
Paso 2	$O(\dots)$
Paso	$O(\dots)$
TOTAL	$O(\dots)$

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Plantilla para el documentar y analizar cada uno de los requerimientos.

Requerimiento 1

Descripción

Calcular la información promedio de los trayectos dado una cantidad de pasajeros, es decir si se elige "1" se buscara todos los trayectos en los que solo hay un pasajero, su salida es el tiempo de ejecución en ms, numero de trayectos que cumplen ese filtro, tiempo promedio de los trayectos, costo total promedio, distancia promedio, costo promedio en peajes, nombre y cantidad

Entrada	Catalog y <i>passenger_count</i> (el número de pasajero que se desea buscar)
Salidas	Retorna un diccionario llamado con el tiempo de ejecución en ms, numero de trayectos que cumplen ese filtro, tiempo promedio de los trayectos, costo total promedio, distancia promedio, costo promedio en peajes, nombre y cantidad del tipo de pago, la cantidad de propina, fecha, o día, de inicio del trayecto con mayor frecuencia de ese tipo de trayectos. Si no hay viajes con ese numero de pasajeros se retorna <code>return {"message": f"No hay trayectos con {passenger_count} pasajeros."}</code>
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Se inicializan contadores y diccionarios (<code>total_*</code> , <code>payment_counter</code> , <code>date_counter</code>)	$O(1)$, a causa de que solo se declara 1 vez las variables
Se recorre la lista de viajes (<code>lt.size(trips)</code>) con un for	$O(N)$
Por cada viaje, se acumula duración, costo, distancia, peajes, propinas, y se actualizan diccionarios	$O(N)$
Se calculan los promedios dividiendo los contadores entre <code>total_trips</code> .	$O(1)$
Se obtiene el método de pago más usado (max sobre <code>payment_counter</code>). En este caso solo hay 4 tipos de métodos de pago, No obstante, la función MAX tiene una complejidad de $O(N)$, W3Schools.com. (2022.). https://www.w3schools.com/python/ref_func_max.asp	$O(4)$
Se obtiene la fecha más frecuente (max sobre <code>date_counter</code>)	$O(31)$
Se toma el tiempo final con <code>get_time()</code> y se calcula con <code>delta_time()</code>	$O(1)$
Se construye el diccionario de salida con los resultados	$O(1)$
TOTAL	$O(N)$

Análisis

La función recorre cada recorrido o trayecto en taxi para verificar cual de estos cumplen el filtro de número de pasajeros, por este motivo su complejidad es $O(N)$ ya que necesita hacer una comparación de que el recorrido posea el numero de pasajeros que se solicita. Asimismo, el cálculo de promedios y el uso de max en estructuras pequeñas (métodos de pago, fechas únicas) no cambia la complejidad global, que se mantiene en $O(N)$. Ahora bien, se podría mejorar la función si previamente en la carga de datos se ordena cuantos pasajeros se tiene para cada distinto viaje.

Requerimiento 2

Descripción

Calcular la información promedio de los trayectos dado un método de pago específico. Se deben reportar los viajes totales, promedios de duración, costo, distancia, peajes, propinas, el número de pasajeros más frecuente y la fecha de finalización más frecuente

Entrada	Catalog y Método de pago
Salidas	Retorna un diccionario llamado retorno que contiene tiempo, total de trayectos, duración promedio, costo promedio, distancia promedio, costo de peajes promedio, cantidad y número de pasajeros, propina promedio y la fecha más frecuente.
Implementado (Sí/No)	Sí se implementó, lo hizo Andrés Zamora

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Se recorren los trayectos de los taxis en catalog, y los que cumplen con el filtro se guardan en met. Se usa un for para el recorrido.	$O(N)$
Se crean variables para almacenar el total de duración, costo, distancia, peajes y propinas. Se crean 2 diccionarios para almacenar la información de cantidad y número de pasajeros, y para las fechas y que tantas veces ocurren.	$O(1)$
Se recorre la lista met con un for y se almacena la información en las variables creadas en el paso anterior.	$O(N)$
Se dividen las variables del segundo paso (que ya tienen la totalidad de la información cargada) en la longitud de la lista met para así sacar su promedio.	$O(1)$
En el caso de los diccionarios de cantidad de pasajeros y de fechas, cada uno hace su debido	$O(N)$

recorrido y comparación para encontrar cual es el mayor.	
Se toma el tiempo final con get_time() y se toma el tiempo final con delta_time()	O(1)
Se crea el diccionario retorno con toda la información necesaria.	O(1)
TOTAL	O(N)

Análisis

Escala de forma lineal por lo que no afecta demasiado el tamaño del csv que se quiera analizar, pero podría mejorarse en memoria simplificando los for y dejando de usar la lista met.

Requerimiento 3

Descripción

Calcular la información del promedio de los trayectos dado un rango de pagos de peajes/dado un rango de pago total.

Entrada	Catalog, Valor menor del precio total pagado a filtrar, Valor mayor del precio total pagado a filtrar
Salidas	Retorna un diccionario llamado con el tiempo de ejecución en ms, numero de trayectos que cumplen ese filtro, tiempo promedio de los trayectos, costo total promedio, distancia promedio, costo promedio en peajes, numero y cantidad de pasajeros mas frecuentes en los trayectos del rango, cantidad de propina promedia, fecha de finalización del trayecto.
Implementado (Sí/No)	NO se implementó, A causa de que no hay estudiante 3

Requerimiento 4

Descripción

Calcular la combinación de barrios origen-destino que tienen el mayor o menor costo total en promedio en un rango de fechas dado. Es decir, se tiene en cuenta únicamente trayectos que tengan barrios de origen y destino diferentes.

Entrada	Catalog, <i>filtro_costo(MAYOR O MENOR)</i> , <i>fecha_inicio</i> , <i>fecha_fin</i>
Salidas	Retorna un diccionario con, tiempo de ejecución en ms, filtro aplicado (MAYOR o MENOR), total de trayectos en el rango de fechas, barrio de origen y barrio de destino seleccionados, promedio de distancia recorrida, promedio de duración, promedio de costo y mensaje en caso de no encontrar trayectos.
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta y Andres Zamora

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Se inicializan las fechas de entrada de los parámetros de la función	$O(1)$,
Se define la función <code>haversine</code> y la función <code>find_neighborhood</code>	$O(1)$
Se recorre la lista de viajes (<code>lt.size(trips)</code>) con un <code>for</code>	$O(N)$
Para cada viaje verificar la fecha de inicio	$O(1)$
Para cada viaje válido: calcular barrios origen y destino con <code>find_neighborhood</code> (recorre lista de barrios de tamaño M)	$O(M)/O(1)$, es una lista finita según el Excel del csv, por lo que se puede considera $O(1)$
Acumular costo, distancia y duración en las combinatorias	$O(1)/O(K)$ (donde K es el número de pares origen y destino válidos)
Seleccionar el mayor o menor costo promedio con <code>max</code> o <code>min</code>	$O(1)/O(K)$
Se toma el tiempo final con <code>get_time()</code> y se calcula <code>delta_time()</code>	
Se construye el diccionario de salida con los resultados	$O(1)$
TOTAL	$O(N*M)$ este es el caso en el que todos los trayectos son válidos, suceden el inicio de

	<i>los recorridos en un barrio y su destino en otro barrio. $O(N)$</i>
--	---

Análisis

En primer lugar, por cada viaje (N) se llama a find_neighborhood dos veces (origen y destino). Asimismo, esa operación se implementa recorriendo todos los barrios disponibles y calculando la distancia con Haversine, lo que implica una complejidad de $O(N \cdot M)$, siendo N el número de viajes y M el número de barrios. Igualmente, una opción para disminuir la complejidad es precalcular y almacenar el barrio de origen y destino de cada viaje durante la carga de datos, de modo que la consulta se reduzca a un simple filtrado temporal y agregación.

Requerimiento 5

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Identificar la franja horaria (por hora de inicio de viaje) con el mayor o menor costo promedio dentro de un rango de fechas. Reporta estadísticas de costo promedio, número de trayectos, duración promedio, promedio de pasajeros, y el viaje más caro y barato de la franja horaria.

Entrada	Catalog, filtro (MAYOR O MENOR), fecha de inicio y fecha final.
Salidas	Un diccionario retorno que contiene tiempo, filtro, el total de trayectos filtrados, la franja horaria, el costo promedio, cantidad de viajes que ocurrieron en la franja horaria, trayecto más y menos costoso en la franja horaria.
Implementado (Sí/No)	Sí, Andrés Zamora y John Acosta

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con get_time()	$O(1)$
Se transforma la fecha inicial y la fecha final de los parámetros a datetime	$O(1)$
Se crea el diccionario horas para guardar la información de cada franja horaria, de tal forma que las llaves son la hora y después cada valor será la información de posible retorno de cada franja horaria (tiempo, filtro, el total de trayectos filtrados, la franja horaria, el costo promedio, cantidad de viajes que ocurrieron en la franja horaria, trayecto más y menos costoso en la franja horaria)	$O(1)$

Se crea la variable trayectos_filtrados para contar cuantos trayectos filtrados por fecha hay en total.	O(1)
Se hace un for para recorrer el diccionario horas, guardando en cada franja horaria la información necesaria para el retorno de cada franja horaria (tiempo, filtro, el total de trayectos filtrados, la franja horaria, el costo promedio, cantidad de viajes que ocurrieron en la franja horaria, trayecto más y menos costoso en la franja horaria), los valores que después deben retornarse como promedio se guardan en forma de suma total.	O(N)
Se hace un for al diccionario horas para recorrer cada franja horaria y calcular los promedios de la información que estaba en forma de suma total, dividiendo las sumas sobre la cantidad de trayectos en esa hora. Pero debido a que siempre serán máximo 24 llaves (por 24 franjas horarias) entonces es O(1)	O(1)
Se usa el filtro del parámetro para elegir la mayor o menos franja horaria en términos de costo y la guardamos en una variable llamada hora_seleccionada	O(1)
“Partimos” la variable hora seleccionada entre franja y datos	O(1)
Arreglamos la franja para que cumpla con la forma [x-y)	O(1)
Creamos el diccionario retorno para guardar toda la información.	O(1)
Retornamos retorno	O(1)
TOTAL	O(N)

Análisis

Muy eficiente porque solo recorre N 1 vez y tiene uso de espacio constante. Escala perfectamente incluso para archivos grandes.

Requerimiento 6

Descripción

Calcular el método de pago más usado y el método de pago que más recaudó iniciando en un barrio en específico en un rango de fechas dada. El algoritmo recorre los viajes, valida si están en el rango, busca el barrio de origen y destino más cercano a las coordenadas dadas con la función de Haversine, acumula métricas (distancia, duración, pagos), y al final retorna estadísticas agregadas como el promedio de distancia y duración, el barrio más visitado y los métodos de pago más usados o con mayor recaudo.

Entrada	Catalog, <i>barrio_inicio</i> , <i>fecha_inicio</i> , <i>fecha_fin</i>
Salidas	Retorna un diccionario con, tiempo de ejecución en ms, total de trayectos en el rango de fechas, promedio de distancia recorrida, promedio de duración, promedio de costo y nombre del barrio mas visitado, tipo de pago, cantidad de trayecto con ese método de pago, precio promedio pagado con ese método, indicar si es el mas usado o no, el que mas recaudo, tiempo promedio con ese método de pago
Implementado (Sí/No)	Sí se implementó, lo hizo John Acosta y Andres Zamora

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Se toma el tiempo inicial con <code>get_time()</code>	$O(1)$
Se inicializan las fechas de entrada de los parametros de la función	$O(1)$,
Se recorre la lista de viajes (N elementos)	$O(N)$
Para cada viaje, se busca barrio de origen y destino con <code>nearest_neighborhood</code>	$O(N*M)$
Se acumulan distancias, duraciones y se actualizan contadores de destinos y pagos	$O(N)$
Se calculan promedios dividiendo acumulados entre la cantidad de viajes válidos	$O(1)$
Se obtiene el barrio más visitado con <code>max(dest_counter)</code>	$O(1)/O(K)$ (donde K es el número de pares origen y destino)
Se procesa la información de medios de pago (max sobre diccionario pequeño)	$O(1)/O(K)$
Se toma el tiempo final con <code>get_time()</code> y se calcula <code>delta_time()</code>	

Se construye el diccionario de salida con los resultados	$O(1)$
<i>TOTAL</i>	<i>$O(N*M)$</i>

Análisis

Inicialmente, como se observó en el requerimiento 4, el paso más costoso es la búsqueda del barrio más cercano (nearest_neighborhood) para cada origen y destino de los viajes, ya que recorre todos los barrios disponibles (M) por cada trayecto (N). Esto genera un costo total de $O(N \cdot M)$, lo cual puede ser lento si el csv contiene muchos viajes. Las operaciones posteriores, como calcular máximos en los diccionarios de destinos o métodos de pago, no impactan tanto porque la cantidad de claves (K o P) es mucho menor que el número de viajes. Ahora bien, para optimizar el código, también sería posible precalcular los barrios de origen y destino al momento de cargar los datos, evitando recalcular distancias en cada consulta. Asimismo, un índice por fecha permitiría filtrar viajes en $O(\log N)$ en lugar de recorrer toda la lista.